GoServer

Project team

October 17, 2026

Contents

1 Project sketch **1**
- 1.1 Problem . . . 1
- 1.2 Scenarios and tasks . . . 1
- 1.3 Decisions recorded . . . 1
- 1.4 Open questions . . . 2

2 Requirements argument **3**
- 2.1 Status . . . 3

3 First-draft observable requirements **4**
- 3.1 Actors . . . 4
- 3.2 Actor populations and sizing . . . 4
- 3.3 Primary messages (conceptual) . . . 4
- 3.4 Tasks and scenarios (high-level) . . . 5
- 3.5 Grammars and transport framing . . . 5
- 3.6 Processes and state machines (summary) . . . 5
- 3.7 Observable properties (for acceptance tests) . . . 5
- 3.8 Next review steps . . . 5
- 3.9 Recent activity and retained artifacts . . . 6

4 Reproducible design and build guide **7**
- 4.1 Repository layout (key paths) . . . 7
- 4.2 Wire protocol and protobuf (how to reproduce stubs) . . . 7
- 4.3 Build and run (example minimal steps) . . . 7
- 4.4 Database schema (DuckDB) – canonical tables . . . 8
- 4.5 Game rules & canonical algorithms (sufficient detail to implement) . . . 8
- 4.6 Server architecture and processes . . . 9
- 4.7 Testing, reproducibility, and CI . . . 9
- 4.8 Pointers to code and stubs . . . 9
- 4.9 How to produce the PDF with this content . . . 9

5 Requirements argument **10**

Chapter 1

Project sketch

This is the working design book for **GoServer**. It begins as a sketch and is revised primarily through the Book layer of project chat.

1.1 Problem

We will implement a KGS-like Go server in Go. A required distinguishing feature is first-class n-player Go support (players: black, white, red, green, blue, ...). The server must be as close as reasonably possible to the KGS behavior for match lifecycle, chat, spectators, and persistence, while supporting n-player game semantics.

This document records the evolving, durable decisions needed to create observable requirements and an implementation mapping.

1.2 Scenarios and tasks

Record the named user and system scenarios. As tasks become precise, their generated interaction diagrams and state machines belong here with their provenance.

Initial scenario sketches (to be refined into observable tasks): - Create account, login, and open a lobby. - Create or join an n-player table; support spectators and in-table chat. - Play an n-player game: turn sequencing, move validation, captures, pass/resign, and scoring/endgame. - Persist game history and export SGF where relevant.

These scenario sketches will be expanded once we fix wire protocol and maximum players.

1.3 Decisions recorded

To avoid guessing on transport and player-cap, the following defaults have been accepted for the first draft of observable requirements:

- Wire protocol / client API: WebSocket primary for interactive game traffic, with HTTP+JSON admin and auxiliary endpoints. KGS compatibility is optional and may be added later. - Maximum players per game (n): pragmatic default cap = 6.

The following additional defaults have now been accepted (recorded durably after explicit acceptance):

- N-player rules variant (default): - Turn order: fixed cyclic turn order over the currently alive players (initial order set at game start). When a player is eliminated they are removed from the

rotation. - Captures/elimination: stones captured are removed immediately as in standard Go. By default, players are not auto-eliminated solely by capture; elimination occurs by explicit resignation or timeout. (An optional configuration may enable elimination when a player has no legal moves, but the default is elimination only by resignation/timeout.) - Scoring/end condition: default = last-man-standing wins. As an alternative, a unanimous pass/resign may trigger Tromp-Taylor scoring (stones + territory) split among survivors; this alternative mode is available as a configuration but the canonical default is last-man-standing.

- Persistence and accounts: persistent user accounts, ratings, and game-history (including SGF export) are required. The recommended initial database is DuckDB; SQLite may be used for single-process development deployments.

The user has confirmed these defaults (WebSocket+HTTP primary, max players = 6, the recorded n-player rule defaults, and DuckDB for persistence) so the next step is to produce the first-draft observable requirements and protobuf event schema unless otherwise instructed.

1.4 Open questions

Keep unresolved design questions explicit until the conversation supplies durable answers.

The following questions remain open for a later iteration (they do not block the first-draft observable requirements because defaults were accepted above):

- Board sizes: which additional board sizes beyond 19x19, 13x13, and 9x9 (default) must be supported? (Default set: 19/13/9.) - Time controls: required time control types (byo-yomi, Canadian, absolute per-move limits)? Behavior on timeout (elimination vs pass) needs confirmation for non-default modes. - Matchmaking & lobbies: ranked matchmaking vs. manual tables/rooms and related policies (we will sketch defaults for the first draft; accept before implementation gate). - Spectators/chat moderation: moderation features and rate limits. - AI/bots: whether bots run in-process or as external services. - Fault-tolerance / deployment: single-process vs clustered deployment model and HA expectations. - Observability & metrics: exact telemetry set and SLO/SLA targets.

Other questions (to be asked later as needed): n-player rules variant configuration options (if you want a different canonical default), persistence migration/backup, matchmaking model, capacity/cost constraints, and tests/CI requirements.

Chapter 2

Requirements argument

Capture decisions and alternatives here before promoting observable behavior through the requirements acceptance gate.

At present we have a provisional design intent and defaults accepted for transport (WebSocket + HTTP), max players (6), n-player rule semantics (as recorded above), and persistence (DuckDB recommended). Once any optional variant parameters (board sizes, time controls, etc.) are finalized, we will generate the observable RequirementSpec (actors, actorPopulations, messages, tasks, grammars, processes, properties, and proofs) and the Requirements.proto bytes schema.

2.1 Status

The following chapter contains the first-draft observable requirements (actors, messages, tasks, grammars, and processes) and an accompanying draft Protobuf schema intended to be used as the wire/event envelope for WebSocket JSON or binary framed messages. These artifacts are a draft for review and have not been accepted as the official requirement gate. Do not proceed to implementation mapping until the project owners accept the required observable requirements.

Chapter 3

First-draft observable requirements

This chapter records the initial, reviewable observable requirements suitable for client/server interaction tests, wire-message grammars, and state-machine derivation. They are deliberately concrete but may be revised.

3.1 Actors

- Player: a registered/authenticated human (or bot) participant who may create accounts, create/join tables, play, chat, and spectate. Each Player has a stable user id. - Spectator: a session that watches a table or game; may be anonymous or authenticated. Spectators can chat in spectator channels subject to moderation rules. - Table (logical actor): a lobby table which can host up to N players and multiple spectators. Responsible for game setup and lifecycle. - Game (logical actor): manages board state, turn order, move validation, captures, elimination, scoring, and game end events. - Admin: HTTP-admin role for management tasks (kick, ban, moderation, metrics). Implemented via authenticated HTTP endpoints.

3.2 Actor populations and sizing

- Max concurrent players per Table: 6 (configurable up to an implementation cap). Default supported board sizes: 19x19, 13x13, 9x9. - Persistent user population: unbounded for the draft; scaling subject to deployment decisions.

3.3 Primary messages (conceptual)

- AuthRequest / AuthResponse: login, logout, session tokens. - CreateTable: request with configuration (board_size, max_players, rule_variant, time_control). - JoinTable / LeaveTable: join or leave a table as player or spectator. - TableUpdate: state changes for tables (player list, status: open/in-game/closed). - GameAction: Move, Pass, Resign, OfferDraw, TimeExpended. - GameStateUpdate: authoritative snapshot or incremental delta (board occupancy, turn index, captured stones, players alive/removed, clocks). - ChatMessage: chat events scoped to lobby, table, or game channels. - ErrorResponse: validation or protocol errors.

These messages map directly to the Protobuf envelope in the draft below.

3.4 Tasks and scenarios (high-level)

- Account lifecycle: register, verify, login, logout. - Table lifecycle: create table, set options, invite/join players, start game when table criteria met, or start manually. - Join/leave spectator: subscribe/unsubscribe to GameStateUpdate and ChatMessage streams. - In-game flow: perform GameAction messages; server validates, updates the Game state, emits GameStateUpdate and event messages (MoveMade, StonesCaptured, PlayerEliminated), and persists move history. - Endgame: upon end condition (last-man-standing or unanimous pass/resign followed by Tromp-Taylor scoring if configured), server emits GameEnded event, persists SGF, updates ratings.

3.5 Grammars and transport framing

- Primary transport: WebSocket bidirectional channel per authenticated session. Messages carried as a JSON representation of the Protobuf Envelope or as binary Protobuf depending on client capability. Administrative and auxiliary APIs are HTTP+JSON. - Message envelope: each WebSocket message is an Envelope with request_id (for client-initiated requests), payload (oneof), and timestamp. Server-initiated events are sent with request_id omitted or matching the initiating request if replying.

3.6 Processes and state machines (summary)

- Lobby process: maintains list of open tables and active players; handles CreateTable, Join/Leave, and TableUpdate broadcasts. - Table process: collects players and spectators, tracks readiness, and starts Game process when conditions are satisfied. - Game process: authoritative sequential interpreter of rules; maintains board, turn index, clocks, and player alive flags. On each valid GameAction, it transitions state, computes captures, updates alive status (per configured elimination rules), updates clocks, persists a move record, and broadcasts GameStateUpdate and domain events. - Persistence process: asynchronously persists authoritative move history and SGF upon game end, and updates account history and ratings.

3.7 Observable properties (for acceptance tests)

- Authenticity: only authenticated sessions can perform player actions associated with a user id. - Turn safety: Game process only accepts a Move from the player whose turn it is; concurrent wrong-turn messages are rejected with an error event. - Deterministic rule application: identical sequences of valid moves produce identical board states and captured-stone lists. - Persistence durability: once server acknowledges GameEnded, SGF and game history persist to DuckDB and can be exported. - Spectator consistency: spectators subscribing to a game's GameStateUpdate should be able to reconstruct the authoritative move sequence and board snapshot.

3.8 Next review steps

These observable requirements form the draft RequirementSpec. To proceed to implementation mapping we need an explicit acceptance of this RequirementSpec by the project owners. After acceptance we will produce the Implementation mapping (file layout, Go packages, WebSocket framing code, DB schema, tests, and CI).

3.9 Recent activity and retained artifacts

This project keeps a small durable history of recent, relevant artifacts so future reviewers remember how to reproduce key developer actions and visualizations.

- Converted the example 9x9 demonstration GIF to an MP4 suitable for in-page playback. Project-relative path: spec/images/9x9_run.mp4. A conversion script and logs live under spec/images/ (see convert_gif_to_mp4.py, convert_to_mp4.sh, and convert_log.txt). - Source images and generation scripts for n-color boards and example play traces are under spec/images/ (board_9x9_3colors.svg, render_9x9_gif.py, render_random_go_gif.py). These show how to render n-color (three-color) boards; similar scripts can be adapted for additional colors. - The first-draft typed RequirementSpec and protobuf schema are implemented in spec/Requirements.lean and spec/Requirements.proto. The typed spec validates with the project tooling (see spec/Requirements.lean top-level validation report).

Keep these artifacts when "cleaning up history"; they are small, reproducible, and document how to generate visualizations and the MP4 used in reviews. If you prefer, we can archive older conversation logs to conversations/ or delete transient debug files; tell me which of these you want removed.

Chapter 4

Reproducible design and build guide

This chapter summarizes the high-level design, algorithms, message flows, and precise build/run instructions so a reader of the PDF can recreate the software from the repository and the guidance here. The PDF intentionally omits full application source but points to canonical files and scripts in the project tree.

4.1 Repository layout (key paths)

- spec/Requirements.proto – canonical Protobuf API describing WebSocket Envelope, ClientCommand, and ServerEvent messages. - spec/Requirements.lean – typed RequirementSpec used to generate tasks and grammars. - spec/generate_proto.sh – helper script to regenerate Go protobuf stubs (requires protoc + protoc-gen-go). - spec/cmd/server/main.go – example server entrypoint scaffold. - spec/internal/server/server.go – WebSocket handler and session management scaffolding. - spec/internal/game/manager.go – authoritative Game process and state machine logic. - spec/internal/store/store.go – persistence interface and DuckDB integration points. - spec/skeleton_*.go – small skeleton programs and helper implementations useful for bootstrapping. - book/Makefile and book/main.tex – PDF build machinery for this book.

Refer to spec/api_README.md for quick API notes and examples.

4.2 Wire protocol and protobuf (how to reproduce stubs)

1. Install protoc and Go protobuf plugin (protoc-gen-go) in your environment. 2. From the project root run: sh spec/generate_proto.sh - This script invokes protoc on spec/Requirements.proto and writes Go stubs to spec/api/. 3. Clients may connect to the server WebSocket and send either JSON-encoded Envelope messages or binary protobuf frames. The Envelope message defined in spec/Requirements.proto is the canonical source of truth.

4.3 Build and run (example minimal steps)

Assuming a POSIX environment with Go 1.20+, protoc, and DuckDB available:

1. Regenerate protobuf stubs: - sh spec/generate_proto.sh 2. Build the server: - go build -o bin/goserver ./spec/cmd/server 3. Run the server (example): - ./bin/goserver –config spec/config.example.yaml

The example server reads config and listens on an HTTP port for WebSocket upgrades. See spec/cmd/server/main.go for exact flags and environment variables.

4.4 Database schema (DuckDB) – canonical tables

A minimal DuckDB schema sufficient for first-pass operation:

- users (id TEXT PRIMARY KEY, username TEXT UNIQUE, password_hash TEXT, created_at TIMESTAMP) - sessions (token TEXT PRIMARY KEY, user_id TEXT REFERENCES users(id), expires_at TIMESTAMP) - games (id TEXT PRIMARY KEY, created_at TIMESTAMP, board_size INTEGER, max_players INTEGER, variant TEXT, result JSONB, sgf TEXT) - moves (id INTEGER PRIMARY KEY AUTOINCREMENT, game_id TEXT REFERENCES games(id), ply INTEGER, player_index INTEGER, x INTEGER, y INTEGER, action TEXT, created_at TIMESTAMP) - ratings (user_id TEXT PRIMARY KEY REFERENCES users(id), elo INTEGER)

Persist these tables to a file (example: data/goserver.db) and ensure DuckDB connection settings match spec/internal/store/store.go.

4.5 Game rules & canonical algorithms (sufficient detail to implement)

This section documents the canonical algorithms required to implement the n-player Go rules described in the requirements.

1. Board representation: - Represent board as two-dimensional array board[board_size][board_size] with values: -1 empty, 0..(N-1) for player stone indices.

2. Group and liberties detection (capture detection): - Given a coordinate with a stone, discover its connected group using iterative DFS/BFS over orthogonal neighbors that share the same player index. - Track liberties by checking adjacent empty intersections; if liberties==0 the entire group is captured and removed.

3. Move validation: - A move is legal if the target intersection is empty and the move does not produce a suicide for the moving player's group, except where the move captures adjacent opponent groups removing their last liberties. - For n-player variants, treat opponent groups as any group belonging to another player index. - Implement Ko/Tsuperko: the canonical first-draft requires simple situational ko prevention per-game by tracking previous full-board hashes and rejecting moves that repeat a previous board hash. For stronger rule sets, implement positional superko by maintaining a set of hashes (Zobrist recommended) and rejecting repeats.

4. Turn ordering and player elimination: - Maintain players[] list at game start. turn_index cycles modulo current alive count. When a player resigns or times out, mark them eliminated and remove them from alive rotation; subsequent modulo wraps over the reduced set. - If a player has no legal moves but has not resigned, default behavior = allow passing; elimination is only by resignation or timeout in the canonical mode.

5. Scoring and endgame: - Default: last-man-standing. When only one player remains (others resigned or timed out), the remaining player is winner. - Alternative: unanimous pass/resign triggers Tromp-Taylor (stones+territory) scoring. To compute Tromp-Taylor score: sum stones on board owned by each player plus territory using a flood-fill from empty points assigning ownership if the region is bordered exclusively by one player.

6. Clocks and timeouts: - Implement per-player clocks tracked by the Game process. On disconnect, optionally pause or continue per configuration. On timeout, mark player eliminated and advance turn.

4.6 Server architecture and processes

Recreate the following logical structure in code:

- HTTP server + WebSocket upgrader that authenticates sessions (JWT or opaque tokens stored in sessions table). - Session manager: maps session token to user id, enforces authentication and session expiry. - Lobby manager: in-memory index of open tables; persists table creation events to DB optionally. - Table manager: manages players and spectators for a table and spawns a Game process when the table starts. - Game process: single goroutine owning the game state for a table; accepts serialized commands over a channel to guarantee sequential deterministic state changes. - Persistence layer: interface for storing and retrieving users, sessions, games, moves, and ratings implemented with DuckDB via the Go driver.

Key implementation notes: - Use channels and goroutines to isolate per-game state. Each Game process should be single-writer to its state to avoid data races and to make deterministic replay simple. - Use Zobrist hashing to compute board hashes efficiently for ko/superko detection and to persist move history.

4.7 Testing, reproducibility, and CI

- Unit tests: exercise board rules (captures, liberties, ko), turn rotation, and scoring on small board sizes (9x9) with deterministic random seeds where randomness exists. - Integration tests: run an in-process server instance, connect websocket clients (use generated protobuf stubs), perform scripted sequences of moves, and assert final persisted SGF and game result. - Reproducible builds: pin Go version in toolchain and declare protoc and protoc-gen-go versions in the README and spec/generate_proto.sh.

4.8 Pointers to code and stubs

- Protobuf canonical source: spec/Requirements.proto - Generated (example) Go stubs: spec/api/requirements.pb.g
- Server scaffold: spec/cmd/server/main.go and spec/internal/server/server.go - Game manager: spec/internal/game/manager.go - Store interface: spec/internal/store/store.go - Skeleton helpers: spec/skeleton_*.go

4.9 How to produce the PDF with this content

From the project root: - make -C book

This will produce book/main.pdf (and book/main-dark.pdf). The PDF includes these textual instructions and pointers; use the repository tree together with the canonical .proto and skeletons to reconstruct the runnable server.

Chapter 5

Requirements argument

After requirements acceptance, record implementation mappings, transports, repository choices, and validation evidence here.

Implementation decisions (language = Go) are accepted; concrete transport bindings, filenames, runtime APIs, and DB integration details will be specified after the requirements acceptance gate.